

PYTHON FUNCTIONS

120 Python Function Interview Questions with Trainer-Style Answers

Interview preparation • Python 3 • Functions, Scope, Closures, Decorators, Recursion & Functional Programming

1. What is a function in Python?

Answer: A function is a reusable block of Python code defined with `def` (or `lambda` for a small anonymous function). It can accept inputs, perform operations, and optionally return a result.

2. Why do we use functions?

Answer: We use functions to reuse code, avoid repetition, improve readability, divide a program into manageable parts, make testing easier, and isolate logic.

3. What is the difference between a function definition and a function call?

Answer: A function definition creates the function and specifies its name, parameters, and body. A function call executes that function by writing its name followed by parentheses and any required arguments.

4. What is the difference between parameters and arguments?

Answer: Parameters are names listed in a function definition; arguments are the actual values or expressions supplied when the function is called.

5. What happens if a function does not return anything?

Answer: If a function reaches the end without executing `return` with a value, it implicitly returns `None`.

6. What is the difference between `print()` and `return`?

Answer: `print()` displays a value to standard output and returns `None`. `return` sends a value back to the caller and immediately exits the current function.

7. Can a function return multiple values?

Answer: Yes. A function can return multiple values syntactically, usually by returning a tuple: `return a, b`. The caller can unpack them: `x, y = function()`.

8. Are functions first-class objects in Python?

Answer: Yes. Functions are first-class objects, meaning they can be assigned to names, stored in collections, passed as arguments, and returned from other functions.

9. Can you assign a function to a variable?

Answer: Yes. Example: `def greet(): print('Hi'); f = greet; f()`. Both names refer to the same function object.

10. Can a function be passed as an argument?

Answer: Yes. Example: `def apply(f, x): return f(x)`. Calling `apply(square, 5)` passes the function `square` as an argument.

11. How are functions stored in memory?

Answer: In CPython, a function is represented by a function object stored in memory. It contains references to its code object, globals, defaults, closure information, and other metadata. Names point to the function object.

12. What is the scope of variables inside a function?

Answer: Variables created inside a function are normally local to that function. Names are resolved according to Python's LEGB rules unless `global`/`nonlocal` changes the binding behavior.

13. What is variable shadowing?

Answer: Variable shadowing occurs when an inner/local name has the same name as an outer-scope name, temporarily hiding the outer name within the inner scope.

14. Explain local and global variables.

Answer: A local variable is bound inside a function and normally belongs to that function's local scope. A global variable is bound at module level. Inside a function, `global x` explicitly tells Python that assignments to `x` refer to the module-level name.

15. What is the LEGB rule?

Answer: LEGB means Local → Enclosing → Global → Built-in. When Python resolves a name, it searches these scopes in that order.

16. What are positional arguments?

Answer: Positional arguments are matched to parameters by their position. Example: `add(2, 3)` assigns 2 to the first parameter and 3 to the second.

17. What are keyword arguments?

Answer: Keyword arguments are matched by parameter name. Example: `add(a=2, b=3)`. Their order can be different from the parameter order.

18. What are default arguments?

Answer: Default arguments have predefined values used when the caller omits those parameters, e.g. `def greet(name='Guest'): ...`

19. What are variable-length arguments?

Answer: Variable-length arguments allow a function to accept an arbitrary number of positional or keyword arguments using `*args` and `**kwargs`.

20. Difference between `*args` and `**kwargs`.

Answer: `*args` collects extra positional arguments into a tuple. `**kwargs` collects extra keyword arguments into a dictionary.

21. When would you use `*args`?

Answer: Use `*args` when the function should accept any number of positional values, such as a function that sums an arbitrary number of numbers.

22. When would you use `**kwargs`?

Answer: Use `**kwargs` when the function should accept any number of named options, settings, or keyword arguments.

23. Can positional arguments appear after keyword arguments?

Answer: Generally no. Once a keyword argument has been supplied, a later positional argument is invalid. For example, `f(a=1, 2)` causes a `SyntaxError`.

24. What happens if required arguments are missing?

Answer: If a required parameter is missing, Python raises `TypeError` when the function is called.

25. Explain positional-only parameters.

Answer: Positional-only parameters are parameters that must be supplied positionally. They are written before `/` in the definition, e.g. `def f(a, /): ...`

26. Explain keyword-only parameters.

Answer: Keyword-only parameters must be supplied by name. They are written after `*` in the definition, e.g. `def f(*, debug=False): ...`

27. Why are mutable default arguments dangerous?

Answer: Mutable defaults such as `[]` or `{}` are created once when the function is defined and reused across calls. Changes can therefore unexpectedly persist between calls.

28. How would you fix the mutable default argument problem?

Answer: Use `None` as the default and create the mutable object inside the function: `def f(items=None): items = [] if items is None else items.`

29. What is a nested function?

Answer: A nested function is a function defined inside another function.

30. Why would you define a function inside another function?

Answer: Nested functions are useful for hiding helper logic, creating closures, implementing decorators, and keeping implementation details local to an outer function.

31. Can an inner function access outer function variables?

Answer: Yes. An inner function can read variables from enclosing functions because of lexical scoping. It needs `nonlocal` if it must rebind an enclosing variable.

32. Explain lexical scoping.

Answer: Lexical scoping means the meaning of a name is determined by the textual nesting of function definitions. Python searches the enclosing scopes when resolving free variables.

33. What is the lifetime of a nested function?

Answer: A nested function object can continue to exist after the outer function returns if something still references it, such as a returned function or a registered callback.

34. Can a nested function be returned?

Answer: Yes. Returning an inner function is a common way to create a closure.

35. What is a closure?

Answer: A closure is a function together with references to variables from an enclosing scope that it needs, allowing it to remember those values after the enclosing function has finished.

36. What conditions are required to create a closure?

Answer: Typically, an inner function must reference a variable from an enclosing non-global scope, and the inner function must survive after that enclosing call, usually through a returned/stored reference.

37. How does a closure differ from a regular nested function?

Answer: A regular nested function is merely defined inside another function. A closure specifically retains access to one or more enclosing-scope variables after the outer function has returned.

38. Explain the role of free variables in closures.

Answer: Free variables are names used by a function that are not local to it and are found in an enclosing scope. Closures retain references to these values.

39. What is the purpose of `nonlocal`?

Answer: `nonlocal` tells Python that an assignment inside a nested function should rebind a variable in the nearest enclosing function scope rather than create a new local variable.

40. Difference between `global` and `nonlocal`.

Answer: `global` refers to a module-level name. `nonlocal` refers to a name in an enclosing function scope; it cannot refer directly to the module/global scope.

41. Where are closures used in real-world applications?

Answer: Closures are useful for decorators, callbacks, configuration, factories, caching, stateful helpers, authentication wrappers, and rate limiters.

42. How are closures used in caching?

Answer: A closure can hold a cache dictionary in its enclosing scope. Each call checks the dictionary and stores results for future calls, without requiring a class.

43. How are closures used in authentication systems?

Answer: A closure can capture authentication-related configuration, such as allowed roles or a token-checking function, and return a wrapper that verifies authorization before calling the protected function. Real systems should use secure credential handling rather than storing secrets casually in closures.

44. How are closures used in rate limiting?

Answer: A closure can retain state such as recent request timestamps or a counter. The returned function checks that state before allowing another call.

45. What is a higher-order function?

Answer: A higher-order function is a function that takes one or more functions as arguments, returns a function, or both.

46. Give examples of higher-order functions in Python.

Answer: Examples include `map()`, `filter()`, `sorted(key=...)`, `functools.reduce()`, decorators, and user-defined functions that accept or return other functions.

47. Why are functions called first-class citizens?

Answer: Functions are called first-class citizens because they are treated like ordinary values: they can be assigned, stored, passed, returned, and created at runtime.

48. Explain `map()`.

Answer: `map(function, iterable)` applies a function to each item and returns a lazy map iterator. Example: `list(map(str.upper, ['a','b'])) → ['A','B']`.

49. Explain `filter()`.

Answer: `filter(function, iterable)` yields items for which the function returns a truthy result. Example: `list(filter(lambda x: x % 2 == 0, range(6))) → [0, 2, 4]`.

50. Explain `reduce()`.

Answer: `functools.reduce(function, iterable)` repeatedly combines items to produce one accumulated result. Example: `reduce(lambda a,b: a+b, [1,2,3]) → 6`. Import it with `from functools import reduce`.

51. Explain `sorted()` with a custom key.

Answer: `sorted(iterable, key=..., reverse=...)` returns a new sorted list. Example: `sorted(words, key=len)` sorts strings by length without changing the original collection.

52. When should you use `map()` instead of loops?

Answer: Use `map()` when you have a simple transformation that reads naturally as applying one function to every item. A comprehension is often more readable for many transformations, especially when conditions are involved.

53. Why is `filter()` considered functional programming?

Answer: `filter()` is associated with functional programming because it treats a function as an input and transforms an iterable by selecting elements according to that function.

54. Compare list comprehensions with `map()`.

Answer: List comprehensions are often more readable and flexible in Python, especially with conditions. `map()` can be concise when an existing function is applied directly. Both can be lazy only in different ways: `map()` returns an iterator, while a list comprehension immediately creates a list.

55. What is a decorator?

Answer: A decorator is a callable that takes a function or class, adds or changes behavior, and returns a replacement callable. The `@decorator` syntax applies it at definition time.

56. Why do we use decorators?

Answer: Decorators provide reusable behavior such as logging, authorization, timing, caching, validation, retries, and registration without rewriting the target function.

57. How do decorators work internally?

Answer: Conceptually, `@decorator def f(...): ...` becomes `f = decorator(f)`. The decorator usually returns a wrapper function that performs extra work and then calls the original function.

58. Explain the relationship between decorators and closures.

Answer: Decorators and closures are closely related because a decorator commonly defines a wrapper inside the decorator function. The wrapper closes over the original function and sometimes configuration values.

59. What is `functools.wraps` and why is it used?

Answer: `functools.wraps` copies useful metadata such as `__name__`, `__doc__`, and other attributes from the wrapped function to the wrapper and sets `__wrapped__`. It improves introspection and debugging.

60. What happens if `wraps` is not used?

Answer: Without `wraps`, the decorated function often appears to have the wrapper's name and docstring, which can confuse documentation tools, debugging, testing, and introspection.

61. How do decorators accept arguments?

Answer: Use another function to create the decorator and capture configuration in an enclosing scope. Typical structure: `def decorator(arg): def real_decorator(func): def wrapper(*args, **kwargs): ...; return wrapper; return real_decorator`.

62. Explain decorator chaining.

Answer: Decorator chaining means applying multiple decorators to one function. `@A` above `@B` means the effective construction is `A(B(function))`; the bottom decorator is applied first.

63. How are decorators used in Flask?

Answer: In Flask, decorators such as route decorators register functions as URL handlers. Custom decorators are also commonly used for authentication, authorization, logging, or validation.

64. How are decorators used in FastAPI?

Answer: In FastAPI, decorators such as `@app.get()` register path operations. Custom wrappers/dependencies can enforce authentication, authorization, logging, or other cross-cutting behavior.

65. How are decorators used in Django?

Answer: In Django, decorators can protect views or enforce conditions, such as login requirements, HTTP method restrictions, or custom permissions. Django also has middleware and other mechanisms for cross-cutting concerns.

66. How would you implement API rate limiting using decorators?

Answer: Use a decorator that tracks request timestamps keyed by client identity, rejects requests exceeding a configured limit, and then calls the target function. In production, distributed systems usually use a shared store such as Redis rather than process-local closure state.

67. How would you implement role-based access control using decorators?

Answer: Use a decorator that checks the caller's authenticated identity/roles before executing the function. It should fail closed and obtain identity from a trusted authentication mechanism rather than an untrusted argument.

68. What is recursion?

Answer: Recursion is a technique where a function calls itself, directly or indirectly, to solve a problem by reducing it to smaller instances of the same problem.

69. What is a base case in recursion?

Answer: A base case is the condition that stops recursive calls and returns a result without making another recursive call.

70. Why is the base case important?

Answer: The base case prevents infinite recursion and provides the smallest problem whose answer is known directly.

71. Difference between recursion and iteration.

Answer: Recursion solves a problem through self-calls; iteration uses loops. Recursion can be elegant for trees and divide-and-conquer problems, while iteration often uses less call-stack memory.

72. When should recursion be avoided?

Answer: Avoid recursion when the depth may be large, when a loop is simpler, or when Python's recursion limit could be reached. Iteration is often preferable for straightforward repetition.

73. What is stack overflow in recursion?

Answer: Python raises `RecursionError` when recursive calls exceed the interpreter's recursion limit. In lower-level terms, excessive recursion can also consume call-stack resources.

74. Explain recursion memory usage.

Answer: Each active recursive call has its own stack frame containing local state and references. Therefore memory usage generally grows with recursion depth, often $O(\text{depth})$.

75. Why doesn't Python optimize tail recursion?

Answer: Python intentionally does not generally perform tail-call optimization. This preserves useful tracebacks and debugging behavior and avoids changing expected stack semantics.

76. What is the recursion limit in Python?

Answer: Python has a recursion limit exposed by `sys.getrecursionlimit()`. The exact default is implementation/version dependent; it is commonly around 1000 in CPython, but code should query it rather than assume a fixed value.

77. Where is recursion used in file systems?

Answer: File systems are naturally hierarchical, so recursive directory traversal can process a directory and then recursively process each subdirectory.

78. Where is recursion used in web crawlers?

Answer: Web crawlers can recursively follow links or traverse hierarchical URL structures, though production crawlers usually need explicit queues, visited sets, depth limits, and rate controls.

79. Where is recursion used in compilers?

Answer: Compilers use recursion for hierarchical structures such as abstract syntax trees, expressions, nested scopes, and grammar structures.

80. Why are decorators usually implemented using closures?

Answer: Closures make it convenient for a decorator's wrapper to retain a reference to the original function and configuration while keeping that state private. However, decorators do not require closures.

81. Can a decorator work without a closure?

Answer: Yes. A decorator can return a callable object or use a class with `__call__` instead of a closure. Closures are simply a convenient implementation technique.

82. Is every closure a nested function?

Answer: No. A closure can be implemented by mechanisms that preserve an enclosing environment; in normal Python practice, closures are commonly created by nested functions, but the defining property is retained access to free variables.

83. Is every nested function a closure?

Answer: No. A nested function becomes a closure when it captures free variables from an enclosing scope. A nested function that captures nothing is just a nested function.

84. Why is the wrapper function typically nested inside a decorator?

Answer: The nested wrapper can capture the original function and decorator configuration through a closure, while keeping implementation details local to the decorator.

85. How do closures retain values after the outer function has finished execution?

Answer: The inner function keeps references to its free variables in its closure. Those captured objects remain alive as long as the closure keeps references to them.

86. How would you implement a cache without using a class?

Answer: Define an outer function containing a dictionary cache and return an inner function that checks and updates that dictionary. This provides stateful caching without a class.

87. How would you count API calls using closures?

Answer: Create an outer function with a counter variable, return an inner function that uses nonlocal counter, increments it, and then calls the target function. This keeps the count private to the closure.

88. How would you log function execution times?

Answer: Create a decorator that records `time.perf_counter()` before and after calling the target, computes elapsed time, and logs it in a finally block so execution is recorded even if the function raises.

89. How would you secure sensitive functions?

Answer: Use a decorator or wrapper that verifies authentication and authorization before executing the sensitive function. Avoid logging secrets, use trusted identity sources, and fail safely when authorization information is missing.

90. How would you create reusable validation logic?

Answer: Create a reusable decorator or higher-order validation function that checks arguments against rules before calling the target. For complex APIs, dedicated validation libraries or type-based validation can be clearer.

91. How would you implement retry logic for failed API calls?

Answer: Create a decorator that attempts the function several times for transient failures, optionally waits between attempts, and only retries exceptions that are safe and expected to be transient. Avoid blindly retrying non-idempotent operations.

92. How would you implement rate limiting?

Answer: Implement a stateful limiter that tracks timestamps/counters and rejects calls above a threshold. For multiple application processes, use shared external state rather than a local closure.

93. How would you implement memoization?

Answer: Memoization caches function results based on arguments. A closure can hold a dictionary keyed by arguments and return a cached result when available. `functools.lru_cache` is the standard built-in-style solution for many cases.

94. How would you build a plugin system using higher-order functions?

Answer: Maintain a registry mapping plugin names to callable functions. A higher-order registration function can accept a plugin and store it, while another function invokes registered plugins. This enables extensibility without hard-coding every plugin.

95. How would you create configurable decorators?

Answer: Use a decorator factory: the outer function accepts configuration, returns a decorator, and that decorator returns a wrapper. Example concept: `@retry(3)` or `@require_role('admin')`.

96. What are pure functions?

Answer: A pure function depends only on its explicit inputs and produces no observable side effects. Calling it with the same inputs should give the same result, assuming the relevant external state does not change.

97. What are impure functions?

Answer: An impure function depends on or changes external state, such as global variables, files, databases, network services, or random state.

98. What are side effects in functions?

Answer: Side effects are observable changes or interactions caused by a function, such as modifying a global/list, writing a file, printing, sending a network request, or updating a database.

99. What is the difference between a named function and an anonymous function?

Answer: A named function has an explicit name, usually created with `def`. An anonymous function is commonly a lambda expression without a normal `def`-defined name.

100. What is a lambda function?

Answer: A lambda is a small anonymous function expression written as `lambda parameters: expression`. Example: `lambda x: x * 2`.

101. When should lambda functions be used?

Answer: Use lambdas for short, simple functions, especially as key functions or callbacks where defining a separate named function would add unnecessary ceremony.

102. What are the limitations of lambda functions?

Answer: Lambdas are limited to a single expression, cannot contain normal statements such as `for/while` blocks or assignment statements, and can become hard to read when overly complex.

103. Can lambda functions have multiple expressions?

Answer: A lambda can contain only one expression, but that expression can include function calls, conditional expressions, comprehensions, and other expression constructs. It cannot contain multiple independent statements.

104. How do lambda functions work with `map()`, `filter()`, and `sorted()`?

Answer: `map()` uses a function to transform items; `filter()` uses a function to select items; `sorted()` can use a key function to compute sorting keys. Lambdas are often used when the function is short and used once.

105. What is function composition?

Answer: Function composition means combining functions so the output of one becomes the input of another. For example, `compose(g, f)` can produce `lambda x: g(f(x))`.

106. What is currying and how is it related to closures?

Answer: Currying transforms a function with multiple arguments into a sequence of single-argument functions. Closures are often used to remember earlier arguments: `curry(a)` returns a function that remembers `a` and waits for `b`.

107. What is partial function application?

Answer: Partial application fixes some arguments of a function in advance, producing a new function with fewer arguments. `functools.partial` is Python's standard tool for this.

108. What is the difference between a closure and a class?

Answer: A closure is primarily a function plus captured enclosing state. A class packages state and behavior through attributes and methods. A closure can be lighter for a small private stateful operation; a class is usually clearer for richer state and multiple operations.

109. When would you choose a closure over a class?

Answer: Choose a closure when the state is small, private, and closely tied to one or a few operations. Choose a class when the state/behavior is substantial, needs multiple methods, inheritance, or clearer object-oriented structure.

110. How does Python handle function arguments internally?

Answer: Python evaluates arguments before entering the function and binds the resulting objects to the function's parameters. Default values are prepared when the function is defined, while argument binding happens on each call.

111. Are arguments passed by value or by reference in Python?

Answer: Python is best described as using call-by-sharing or pass-by-object-reference. The function receives references to the same objects; assigning a parameter to a new object does not change the caller's name, but mutating a shared mutable object is visible to the caller.

112. Explain the concept of pass-by-object-reference.

Answer: Pass-by-object-reference means both caller and callee can refer to the same object. Example: a function that does `items.append(1)` can change the caller's list, but `items = [1]` only rebinds the local parameter.

113. What is argument unpacking?

Answer: Argument unpacking expands an iterable into positional arguments or a mapping into keyword arguments. Example: `f(*[1,2])` and `f(**{'a':1,'b':2})`.

114. How do * and ** work during function calls?

Answer: In a call, `*` unpacks an iterable into positional arguments; `**` unpacks a mapping into keyword arguments. In a function definition, `*args` collects positional extras and `**kwargs` collects keyword extras.

115. Can functions be stored in data structures such as lists and dictionaries?

Answer: Yes. Functions can be stored in lists, tuples, dictionaries, sets when hashable, and other containers. Example: `funcs = [str.upper, str.lower]`.

116. Can a function return another function?

Answer: Yes. A function can return another function, often using a nested function and closure.

117. Can a function accept another function and return a function?

Answer: Yes. A higher-order function can accept a function and return a function. Decorators are a common example.

118. What are callback functions?

Answer: A callback is a function supplied to another function or system so it can be called later when an event, operation, or condition occurs.

119. Where are callback functions used in real-world applications?

Answer: Callbacks are used in event handlers, GUI programming, asynchronous systems, sorting keys, APIs, web frameworks, background tasks, and data-processing pipelines.

120. What is the difference between callbacks and higher-order functions?

Answer: A higher-order function is a broad concept: any function that accepts or returns functions. A callback is specifically a function supplied for later invocation by another component. A callback is therefore a common use of higher-order functions, but the terms are not identical.